QISKIT FALL FEST 2026

Quantum Portfolio Optimization with QAOA

A six-stock statevector study with an exact classical baseline, parameter sweeps, and constraint-aware classical post-processing.

STOCKS	PORTFOLIO SIZE
6	3
Synthetic instance	Exact-cardinality

QAOA DEPTHS	NEW FEATURE
1–4	Q-Repair
Compared	Classical recovery

Research prototype • Reproducible local experiments

Portfolio selection as binary optimization

Balance risk and expected return while selecting exactly K stocks.

Minimize the penalized objective:

$$C(x) = q\, x^{\top}\Sigma x - \mu^{\top}x + \lambda(\Sigma x_i - K)^2$$

$x_i \in \{0,1\}$ • $n = 6$ • $K = 3$ • $q = 0.5$

- Binary decision: select or do not select each stock.
- Risk term: quadratic form using a positive semidefinite matrix.
- Return term: rewards higher expected returns.
- Penalty: discourages portfolios with the wrong number of stocks.

Data is synthetic and generated deterministically with NumPy seed 2026. It is a benchmark instance, not a claim about real market performance.

Establish the reference optimum

Exhaustive search checks every portfolio satisfying the constraint.

ALL BITSTRINGS	FEASIBLE SET	BEST COST	OPTIMAL STOCKS
64	20	–0.503015	2, 3, 6
2■ candidates	Choose 3 of 6	Original objective	One-based indices

Optimal bitstring: 011001

- The exact feasible optimum is the reference for QAOA evaluation.
- Classical enumeration is inexpensive at six stocks.
- For twelve stocks with six selections, there are 924 feasible portfolios.

The 12-stock experiment is a separate classical baseline. No completed 12-stock QAOA result is claimed.

Alternating cost and mixer operations

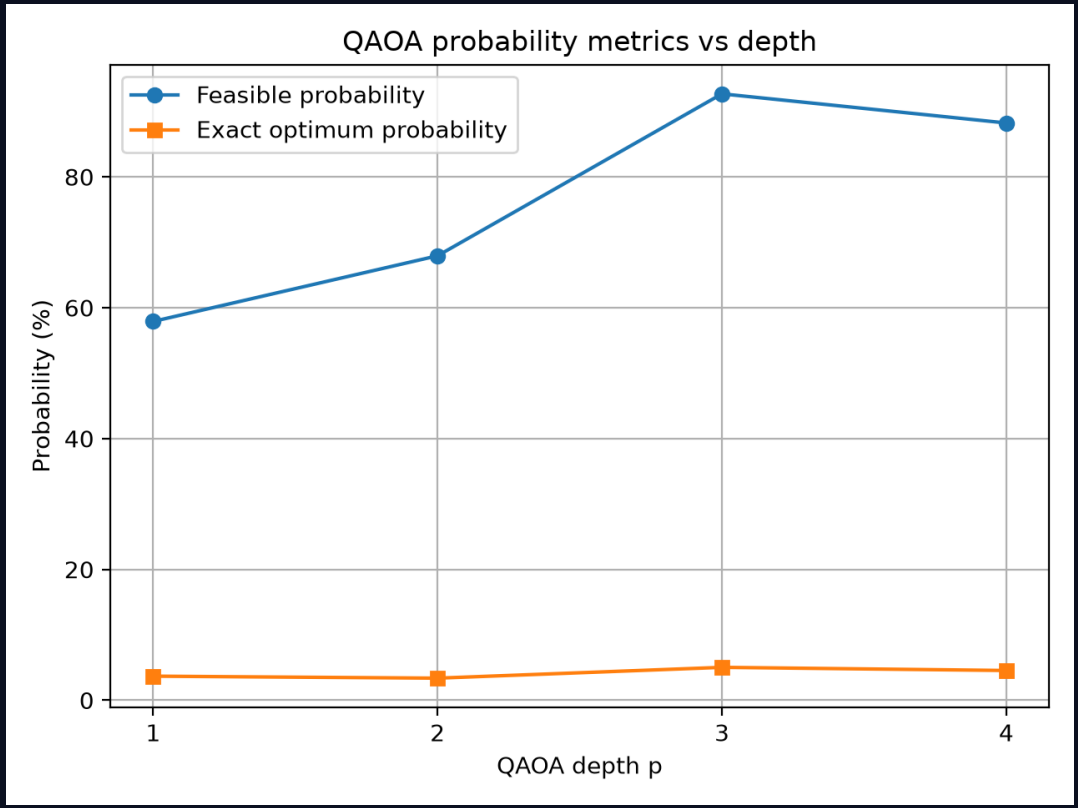
Statevector QAOA experiments with p = 1, 2, 3, and 4.

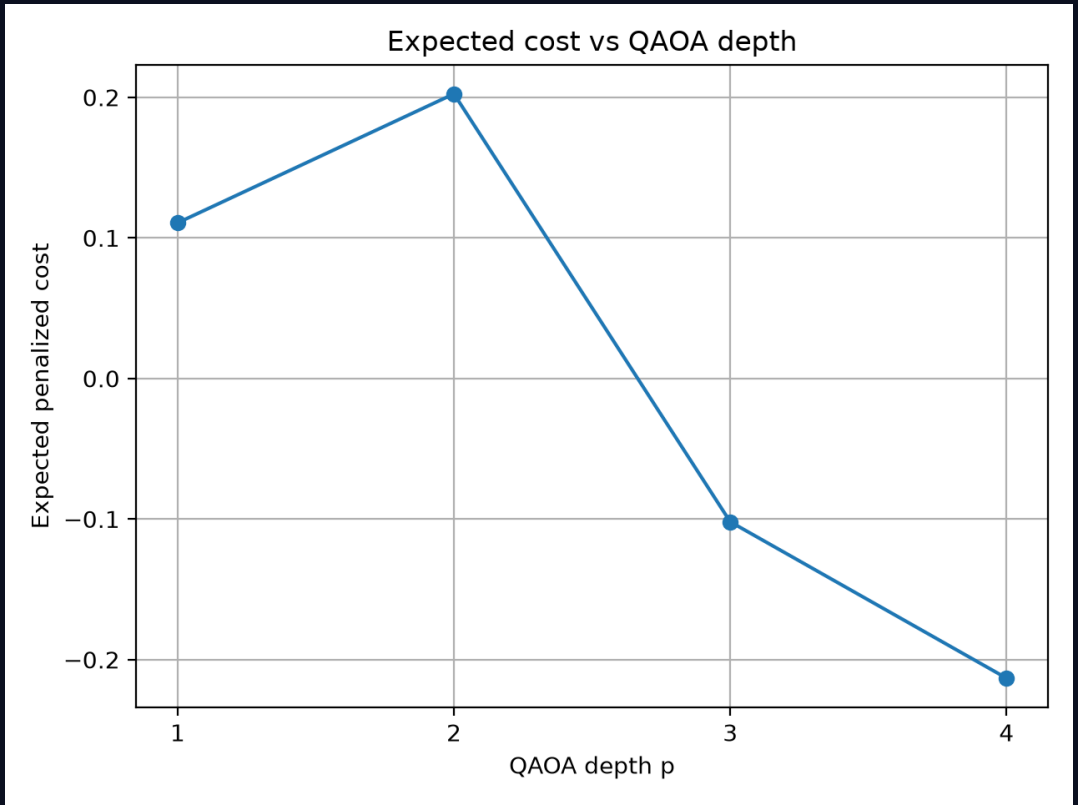
01	**Initialize**	Uniform superposition over 64 bitstrings.
02	**Cost phase**	Apply $\exp(-i\gamma C(x))$ to each basis-state amplitude.
03	**Mixer**	Apply $RX(2\beta)$ to each of the six qubits.
04	**Optimize**	Use COBYLA to minimize expected penalized cost.

Implementation note: the current experiment applies cost phases and single-qubit mixers directly to a statevector. It is not evidence of hardware execution or GPU acceleration.

Depth changes multiple performance metrics

Observed results from the finite-budget QAOA depth sweep.

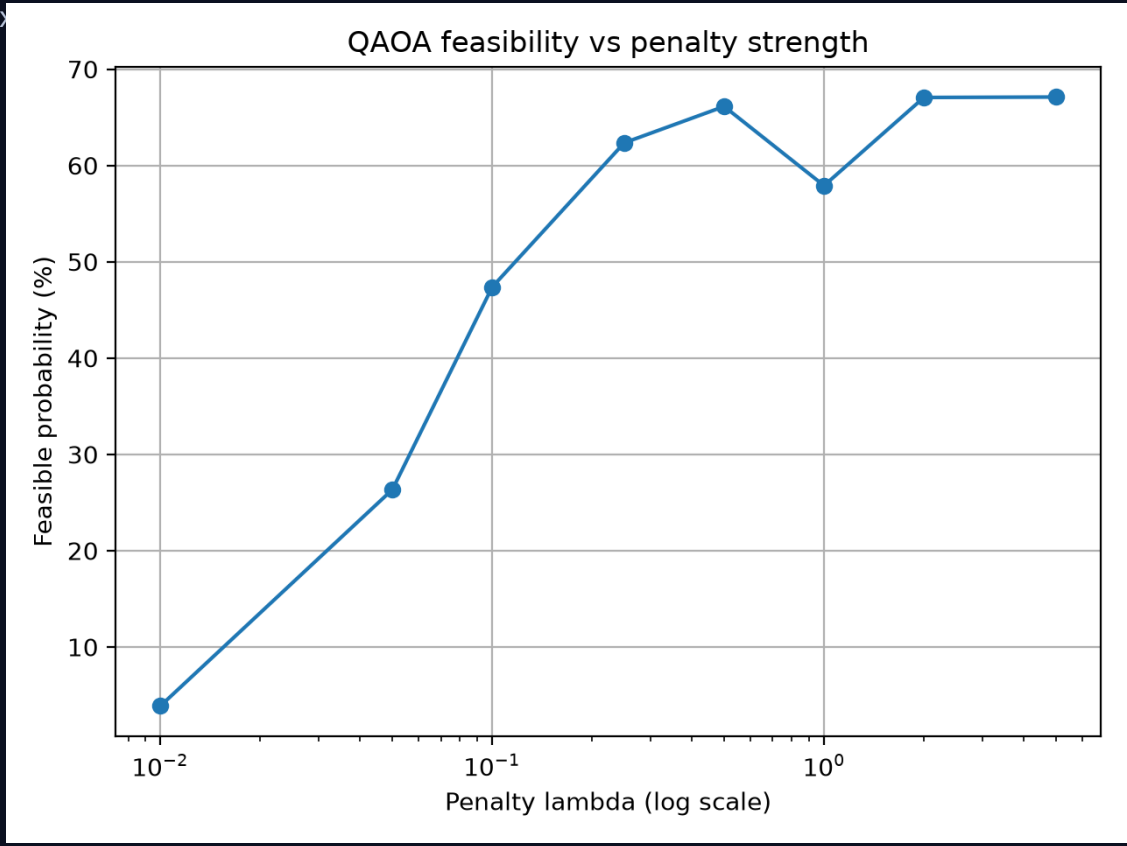




p=3: feasibility 92.69%, optimum probability 5.03%. p=4: lowest observed expected cost (–0.213192). All p=2–4 runs reached the 100-evaluation budget.

Penalty strength is a trade-off

Feasibility and ex



HIGHEST FEASIBILITY

67.10%

λ = 5 in tested sweep

HIGHEST OPTIMUM MASS

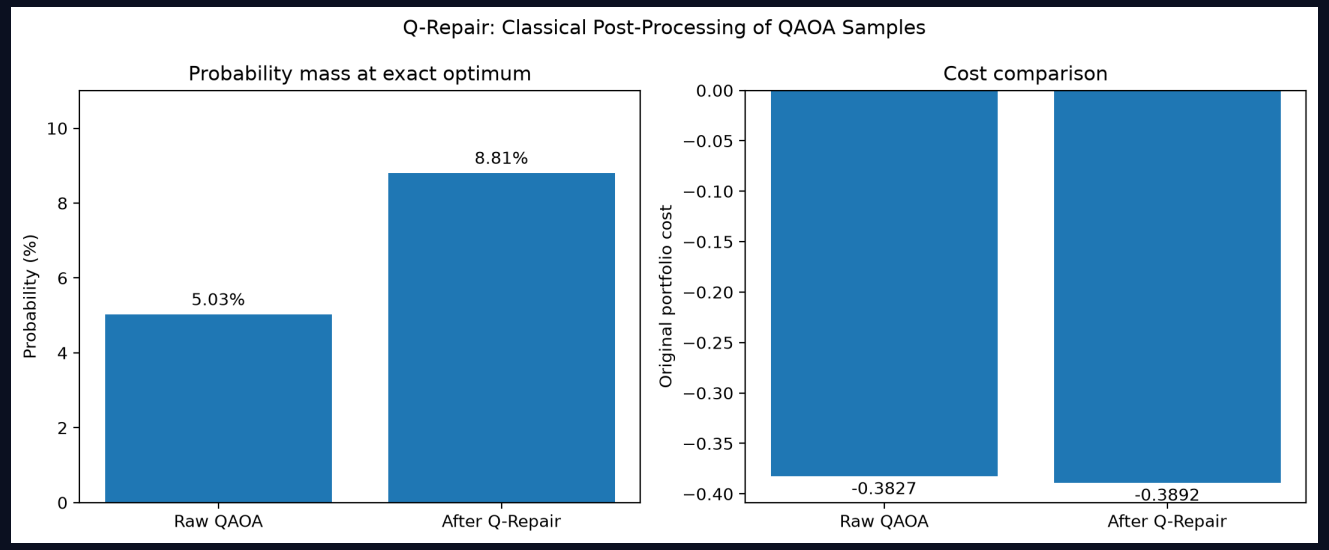
3.68%

λ = 1 in tested sweep

Some optimizer runs returned success=False. The finite search budget and non-monotonic results limit conclusions; these are observed runs, not globally optimized penalty settings.

Q-Repair: constraint-aware post-processing

Map each sampled bitstring to a nearby feasible portfolio.



RAW OPTIMUM PROBABILITY

5.03%

Selected p=3 run

AFTER REPAIR

8.81%

Mapped probability mass

- Repair changes the fewest bits needed to reach exactly three stocks.
- Ties are resolved by the original portfolio objective.
- All eight optimizer starts reached the 100-evaluation limit.
- Classical repair is not quantum advantage.

What we learned

A reproducible optimization study—not a claim of quantum advantage.

- Exhaustive search establishes the six-stock optimum: stocks 2, 3, 6.
- QAOA depth and penalty settings affect feasibility and solution quality differently.
- Q-Repair increased mapped optimum probability mass from 5.03% to 8.81% in the selected run.
- Optimizer initialization sensitivity remains significant under the current budget.
- The 12-stock result is classical; no larger QAOA success is claimed.

Next steps

Improve optimizer convergence, test repeated seeds, compare with a gate-level Qiskit circuit, and evaluate on historical financial data.

Reproducibility: Python • NumPy • SciPy • Qiskit • statevector simulation